

Análisis descriptivo de variables cualitativas

Issue #38 — MA2003B, equipo 7

Tabla de contenidos

0. Carga y construcción de las categóricas	1
periodo	2
tipo_dia y festivo	2
llovio	2
Temporada en el diario	3
1. Distribución de frecuencia	3
2. estacion cruzada con la cobertura temporal	5
3. Mediana en escala ordinal	6
4. Visualización	7
5. Criterios de aceptación	12

Issue #38. Contraparte cualitativa del análisis descriptivo. Opera sobre `sima_limpio_horario.csv` y `sima_limpio_diario.csv`, las dos salidas de `src/limpieza.py` (#13), **no** sobre `sima_transformado_horario.csv`, las variables cuantitativas de ese archivo están en escala Box-Cox estandarizada y las indicadoras de excedencia no viven ahí.

Las figuras se escriben en `reports/figuras/` y las cifras alimentan `reports/secciones/eda_cualitativas.tex`.

0. Carga y construcción de las categóricas

Tres variables en alcance no venían construidas y se generan aquí.

```
import sys
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

# No fijar matplotlib.use("Agg") aquí: es un backend no interactivo, así que
# plt.show() no emite nada, Quarto no captura ninguna figura y el PDF sale con
# los captions vacíos ("Figura 1" sin gráfica). Los savefig() a reports/figuras/
# sí funcionan con Agg, y por eso el problema pasó desapercibido: lo que se
# rompe es el PDF de la notebook, que es lo que hace revisable la PR.
# Con el backend inline del kernel, savefig() escribe el archivo igual y
# show() además incrusta la figura en este documento.
```

```

RAIZ = Path.cwd().parent
PROC = RAIZ / "data" / "processed"
FIGS = RAIZ / "reports" / "figuras"
FIGS.mkdir(parents=True, exist_ok=True)

# El cwd de Quarto es notebooks/, así que la raíz del repo no está en sys.path
# y el import de src/ falla sin esta línea.
sys.path.insert(0, str(RAIZ))

H = pd.read_csv(PROC / "sima_limpio_horario.csv", parse_dates=["fecha"])
D = pd.read_csv(PROC / "sima_limpio_diario.csv", parse_dates=["fecha"])
print(f"horario: {H.shape}    diario: {D.shape}")

```

```

horario: (640583, 38)    diario: (26691, 9)

```

periodo

Binning de la hora en las cuatro fases del ciclo fotoquímico diario. Los cortes se heredan de `archive/02_transformacion_datos.qmd` y no se modifican, para que la variable sea comparable con lo ya reportado: bloques de seis horas, madrugada 00–05, mañana 06–11, tarde 12–17, noche 18–23.

```

ORDEN_PERIODO = ["madrugada", "mañana", "tarde", "noche"]
H["periodo"] = pd.cut(H["hora"], bins=[-1, 5, 11, 17, 23], labels=ORDEN_PERIODO)

```

tipo_dia y festivo

`fin_de_semana` agrupa sábado y domingo pese a que su actividad vehicular difiere, y no separa los días de asueto. `tipo_dia` los separa en cuatro niveles: días de descanso obligatorio de la LFT art. 74, jueves y viernes santo, y el periodo 24–31 de diciembre, que tienen patrón de tráfico anómalo y contaminarían cualquier contraste semanal si quedaran dentro de `laborable`.

La construcción vive en `src/agregacion_diaria.py` (#51) y se importa: el modelado la necesita como predictor, así que dejó de ser código de notebook.

```

from src.agregacion_diaria import marcar_calendario

H = marcar_calendario(H); D = marcar_calendario(D)

```

llovio

Dicotomización de `RAINF`. A diferencia de la versión de #14, **los faltantes se conservan como faltantes**: asignarles 0 los reclasifica como “no llovió” e infla esa clase con horas en las que simplemente no se midió.

```
H["llovio"] = np.where(H.RAINF.isna(), np.nan, (H.RAINF > 0).astype(float))
print("llovio=1:", int((H.llovio == 1).sum()),
      "| sin dato:", int(H.llovio.isna().sum()))
```

llovio=1: 9271 | sin dato: 19360

Temporada en el diario

```
MES_TEMP = {12:"invierno",1:"invierno",2:"invierno",3:"primavera",4:"primavera",
             5:"primavera",6:"verano",7:"verano",8:"verano",
             9:"otoño",10:"otoño",11:"otoño"}
D["temporada"] = D.fecha.dt.month.map(MES_TEMP)
D["fin_de_semana"] = D.fecha.dt.dayofweek.isin([5, 6]).astype(int)
```

1. Distribución de frecuencia

periodo y llovio son horarias; las indicadores de excedencia derivan del MDA8 y son diarias. Si se tabulan juntas, cada valor diario se repite 24 veces y el porcentaje de excedencia deja de coincidir con el publicado en #14.

```
def frecuencia(s: pd.Series, nombre: str) -> pd.DataFrame:
    vc = s.value_counts(dropna=False)
    t = pd.DataFrame({"n": vc})
    t["pct_total"] = (t.n / t.n.sum() * 100).round(2)
    validos = s.notna().sum()
    t["pct_validos"] = np.where(pd.isna(t.index), np.nan,
                                (t.n / validos * 100).round(2))
    print(f"\n=== {nombre} (n={len(s):,}, válidos={validos:,:})")
    print(t.to_string())
    return t

print(f"BLOQUE A - hora-estación (n = {len(H):,:})")
for c in ["temporada", "periodo", "tipo_dia", "fin_de_semana", "llovio"]:
    frecuencia(H[c], c)
```

BLOQUE A - hora-estación (n = 640,583)

```
=== temporada (n=640,583, válidos=640,583)
      n  pct_total  pct_validos
temporada
primavera 161184    25.16    25.16
verano    161184    25.16    25.16
otoño     159432    24.89    24.89
```

invierno	158783	24.79	24.79
----------	--------	-------	-------

```
=== periodo (n=640,583, válidos=640,583)
```

	n	pct_total	pct_validos
periodo			
mañana	160146	25.0	25.0
tarde	160146	25.0	25.0
noche	160146	25.0	25.0
madrugada	160145	25.0	25.0

```
=== tipo_dia (n=640,583, válidos=640,583)
```

	n	pct_total	pct_validos
tipo_dia			
laborable	434879	67.89	67.89
domingo	88728	13.85	13.85
sabado	88392	13.80	13.80
festivo	28584	4.46	4.46

```
=== fin_de_semana (n=640,583, válidos=640,583)
```

	n	pct_total	pct_validos
fin_de_semana			
0	457463	71.41	71.41
1	183120	28.59	28.59

```
=== llovio (n=640,583, válidos=621,223)
```

	n	pct_total	pct_validos
llovio			
0.0	611952	95.53	98.51
NaN	19360	3.02	NaN
1.0	9271	1.45	1.49

```
print(f"BLOQUE B - día-estación (n = {len(D):,})")
for c in ["temporada", "tipo_dia", "excede_anio_1", "excede_anio_3",
          "excede_anio_5", "excede_1h"]:
    frecuencia(D[c], c)
```

BLOQUE B - día-estación (n = 26,691)

```
=== temporada (n=26,691, válidos=26,691)
```

	n	pct_total	pct_validos
temporada			
primavera	6716	25.16	25.16
verano	6716	25.16	25.16
otoño	6643	24.89	24.89
invierno	6616	24.79	24.79

```
=== tipo_dia (n=26,691, válidos=26,691)
```

	n	pct_total	pct_validos
tipo_dia			
laborable	18120	67.89	67.89
domingo	3697	13.85	13.85
sabado	3683	13.80	13.80
festivo	1191	4.46	4.46

```
=== excede_anio_1 (n=26,691, válidos=25,100)
```

	n	pct_total	pct_validos
excede_anio_1			
0.0	22275	83.46	88.75
1.0	2825	10.58	11.25
NaN	1591	5.96	NaN

```
=== excede_anio_3 (n=26,691, válidos=25,100)
```

	n	pct_total	pct_validos
excede_anio_3			
0.0	20864	78.17	83.12
1.0	4236	15.87	16.88
NaN	1591	5.96	NaN

```
=== excede_anio_5 (n=26,691, válidos=25,100)
```

	n	pct_total	pct_validos
excede_anio_5			
0.0	17079	63.99	68.04
1.0	8021	30.05	31.96
NaN	1591	5.96	NaN

```
=== excede_1h (n=26,691, válidos=26,691)
```

	n	pct_total	pct_validos
excede_1h			
0	24497	91.78	91.78
1	2194	8.22	8.22

Verificación contra #14. `excede_anio_1` debe dar 11.25 % sobre válidos (2,825 de 25,100). Sobre el total de días-estación daría 10.58 %; la diferencia es solo la base, y por eso se reportan las dos columnas.

2. estacion cruzada con la cobertura temporal

La frecuencia absoluta de `estacion` mide tiempo de operación, no volumen de información útil. Se reporta junto con la ventana de operación y con el porcentaje de horas que efectivamente tienen O medido.

```
cov = H.groupby("estacion").agg(n=("fecha", "size"),
                                ini=("fecha", "min"), fin=("fecha", "max"))
```

```

cov["horas_teoricas"] = ((cov.fin - cov.ini).dt.total_seconds() / 3600 +
    ↪ 1).astype(int)
cov["completitud_rejilla"] = (cov.n / cov.horas_teoricas * 100).round(1)
cov["pct_filas"] = (cov.n / cov.n.sum() * 100).round(2)
cov["pct_o3_medido"] = H.groupby("estacion").O3.apply(
    lambda s: round(s.notna().mean() * 100, 1))
print(cov.sort_values("n", ascending=False, kind="stable").to_string())

```

	n	ini	fin	horas_teoricas	completitud_rejilla
estacion					
CE	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
NE	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
NE2	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
NE3	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
NO	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
NO2	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
NTE	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
NTE2	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
SE	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
SE2	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
SE3	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
SO	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
SO2	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
SUR	43824	2021-01-01 00:00:00	2025-12-31 23:00:00	43824	100.0
NO3	27047	2022-12-01 01:00:00	2025-12-31 23:00:00	27047	100.0

3. Mediana en escala ordinal

periodo es la única ordinal. Se adopta el orden del ciclo diario sin cerrarlo en círculo: la mediana ordinal no tiene análogo circular limpio como sí lo tiene la codificación seno/coseno usada para hora y mes en #14. Se define como el valor en la posición $\lceil n/2 \rceil$; no se promedian categorías.

```

cod = dict(zip(ORDEN_PERIODO, range(4)))

def mediana_ordinal(s: pd.Series, etiqueta: str) -> str:
    x = np.sort(s.map(cod).dropna().values); n = len(x)
    med = ORDEN_PERIODO[x[int(np.ceil(n / 2)) - 1]]
    dist = np.round(np.bincount(x, minlength=4) / n * 100, 2)
    print(f"[{etiqueta}] n={n:,} -> mediana = {med}")
    print("    distribución%:", dict(zip(ORDEN_PERIODO, dist)))
    return med

mediana_ordinal(H.periodo, "rejilla horaria completa")
mediana_ordinal(H.loc[H.O3.notna(), "periodo"], "filas con O3 válido")

```

```
[rejilla horaria completa] n=640,583 -> mediana = tarde
distribución %: {'madrugada': np.float64(25.0), 'mañana': np.float64(25.0), 'tarde': np.float64(25.0)}
[filas con O3 válido] n=611,601 -> mediana = tarde
distribución %: {'madrugada': np.float64(24.9), 'mañana': np.float64(24.8), 'tarde': np.float64(24.9)}

'tarde'
```

Sobre la rejilla completa el resultado es un artefacto: cuatro bloques equifrecuentes hacen que la posición central caiga sobre la primera observación de **tarde**, a un registro de la frontera con **mañana**. Sobre las filas con O3 medido las frecuencias dejan de ser uniformes —los faltantes se concentran de noche— y la mediana sí queda determinada por los datos. Esa es la cifra que se reporta.

temporada se deja como nominal pura: su orden también es cíclico y no tiene un corte natural que justifique linealizarlo.

4. Visualización

Barras para todo lo que tenga más de dos niveles; pastel solo para las binarias, donde la proporción se lee sin recurrir al eje.

```
plt.rcParams.update({"font.size": 9, "font.family": "serif",
                    "axes.spines.top": False, "axes.spines.right": False,
                    "figure.dpi": 150})
GRIS, AZUL, ROJO, CLARO = "#8c8c8c", "#3b6ea5", "#b5443a", "#d9d9d9"
```

```
c = cov.sort_values("n", kind="stable")
col = [ROJO if e == "N03" else AZUL for e in c.index]
# A una columna del reporte (~3.4 in). Los dos paneles siguen lado a lado
# porque comparten el eje y de estaciones; apilarlos repetiría las 15 etiquetas.
# La nota al pie que llevaba la figura se elimina: el caption del .tex ya dice
# lo mismo sobre N03.
fig, ax = plt.subplots(1, 2, figsize=(3.4, 3.1), gridspec_kw={"width_ratios": [2,
↪ 1]})
ax[0].barh(c.index, c.n, color=col)
for i, v in enumerate(c.n):
    ax[0].text(v + 900, i, f"{v:,}", va="center", fontsize=5.5)
ax[0].set_xlim(0, 52000); ax[0].set_xlabel("Registros hora-estación", fontsize=7)
ax[0].set_title("Frecuencia absoluta", loc="left", fontsize=8.5)
ax[0].tick_params(labelsize=6.5)
ax[1].barh(c.index, c.pct_o3_medido, color=col); ax[1].set_xlim(0, 100)
ax[1].set_xlabel("% horas con O3_3$", fontsize=7); ax[1].set_yticklabels([])
ax[1].set_title("Complejidad O3_3$", loc="left", fontsize=8.5)
ax[1].tick_params(labelsize=6.5)
for a in ax:
    for lado in ("top", "right"):
        a.spines[lado].set_visible(False)
```

```
fig.tight_layout(pad=.3)
fig.savefig(FIGS / "eda_cual_estacion.pdf"); plt.show()
```

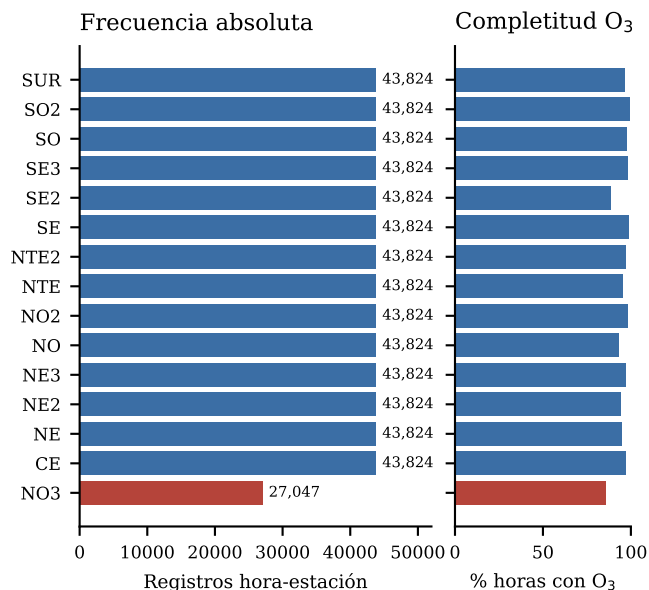


Figura 1: Frecuencia por estación y completitud de O_3 en paralelo.

```
fig, ax = plt.subplots(figsize=(3.4, 2.9)) # a una columna del reporte
cv = cov.sort_index(ascending=False)
for i, (e, r) in enumerate(cv.iterrows()):
    ax.barh(i, (r.fin - r.ini).days, left=r.ini, height=.6,
            color=ROJO if e == "NO3" else AZUL)
ax.set_yticks(range(len(cv))); ax.set_yticklabels(cv.index, fontsize=6.5)
ax.tick_params(axis="x", labelsize=6.5)
ax.set_title("Ventana de operación (2021-01-01 a 2025-12-31)",
            loc="left", fontsize=8.5)
fig.tight_layout(); fig.savefig(FIGS / "eda_cual_cobertura.pdf"); plt.show()
```

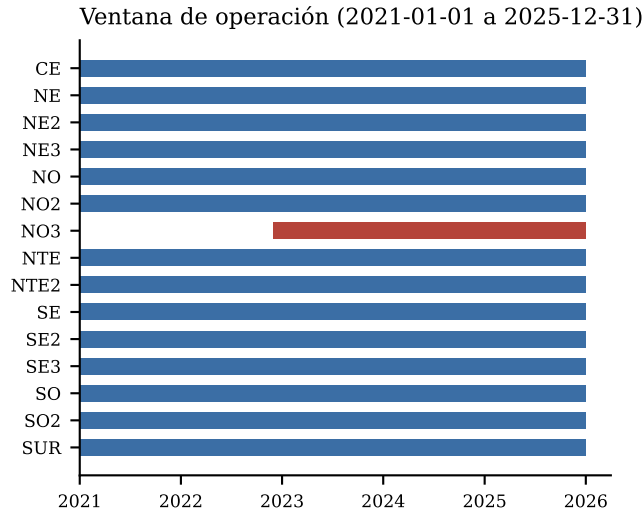



Figura 2: Ventana de operación de cada estación.

```
# Tres paneles apilados y barras horizontales: la figura va a una columna del
# reporte (~3.4 in), no a doble columna. Apilada, la etiqueta de cada nivel cabe
# en el eje y sin rotar, que es lo que permite bajar el tamaño de letra.
fig, axs = plt.subplots(3, 1, figsize=(3.4, 3.5))
paneles = [(H.temperatura, ["invierno", "primavera", "verano", "otoño"], "Temperatura"),
            (H.periodo, ORDEN_PERIODO, "Periodo del
↪ día"),
            (H.tipo_dia, ["laborable", "sabado", "domingo", "festivo"], "Tipo de
↪ día")]
for a, (s, orden, tit) in zip(axs, paneles):
    p = s.value_counts().reindex(orden) / len(s) * 100
    y = range(len(orden))
    a.barh(y, p, color=AZUL, height=.62)
    a.set_yticks(y)
    a.set_yticklabels([o.capitalize() for o in orden], fontsize=7.5)
    a.invert_yaxis()
    for i, x in enumerate(p):
        a.text(x + max(p) * .015, i, f"{x:.1f}", va="center", fontsize=7)
    a.set_xlim(0, max(p) * 1.22); a.tick_params(axis="x", labelsz=7)
    a.set_title(tit, loc="left", fontsize=9)
    for lado in ("top", "right"):
        a.spines[lado].set_visible(False)
axs[2].set_xlabel("% de registros hora-estación", fontsize=7.5)
axs[1].axvline(25, ls="--", lw=.8, color=GRIS)
fig.tight_layout(pad=.4); fig.savefig(FIGS / "eda_cual_nominales.pdf"); plt.show()
```

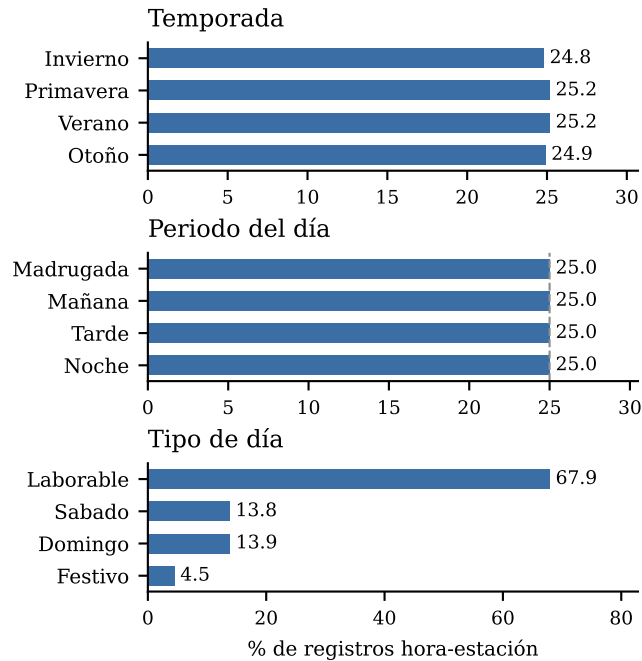


Figura 3: Distribución relativa de temporada, periodo y tipo_día.

```
v = [(H.llovio == 0).sum(), (H.llovio == 1).sum(), H.llovio.isna().sum()]
fig, ax = plt.subplots(figsize=(3.6, 3.6))
ax.pie(v, labels=["No llovió", "Llovió", "Sin dato"], colors=[CLARO, AZUL, GRIS],
      autopct=lambda p: f"{p:.2f}%", startangle=90, textprops={"fontsize": 8},
      wedgeprops={"edgecolor": "white", "linewidth": .8})
ax.set_title(f"llovio (n = {len(H):,} hora-estación)", fontsize=10)
fig.tight_layout(); fig.savefig(FIGS / "eda_cual_llovio.pdf"); plt.show()
```

llovio (n = 640,583 hora-estación)

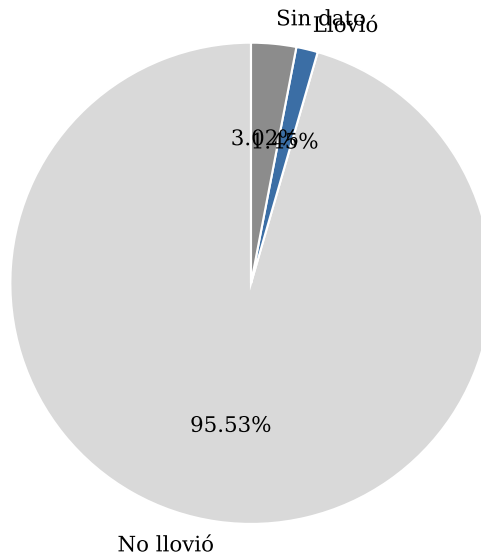


Figura 4: llovio, con la categoría sin dato explícita.

```
# Rejilla 2x2 en vez de 1x3: la figura va a una columna del reporte. Tres pays
# en fila a ~3.4 in de ancho quedarían de 1.1 in y sin espacio para los
# porcentajes; en 2x2 el diametro se conserva y la celda libre aloja la leyenda,
# que así deja de repetirse tres veces.
fig, axs = plt.subplots(2, 2, figsize=(3.4, 3.0))
axs = axs.ravel()
for a, (c_, u) in zip(axs, [("excede_anio_1", 65), ("excede_anio_3", 60),
                           ("excede_anio_5", 51)]):
    s = D[c_]
    cunas, _, _ = a.pie([(s == 0).sum(), (s == 1).sum()],
                        colors=[CLARO, ROJO], startangle=90, autopct=lambda p: f"{p:.2f}%",
                        textprops={"fontsize": 6.5},
                        wedgeprops={"edgecolor": "white", "linewidth": .8})
    a.set_title(f"{u} ppb", fontsize=8.5)
axs[3].axis("off")
axs[3].legend(cunas, ["No excede", "Excede"], loc="center", fontsize=7.5,
              frameon=False)
fig.suptitle("Balance de clases del MDA8\n"
             f"(n = {int(D.excede_anio_1.notna().sum())}, día-estación válidos)",
             fontsize=8.5)
fig.tight_layout(pad=.3)
fig.savefig(FIGS / "eda_cual_excedencias.pdf", bbox_inches="tight"); plt.show()
```

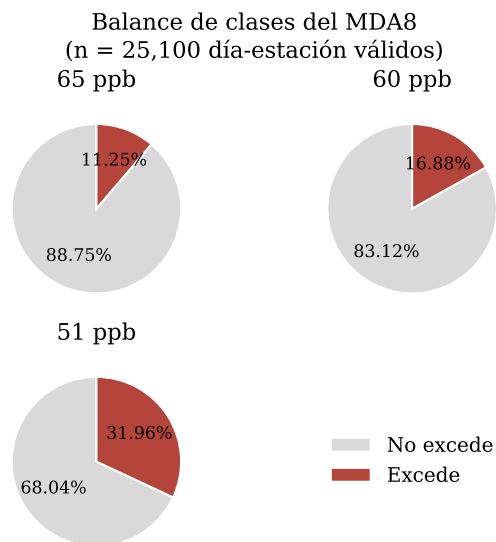


Figura 5: Balance de clases del MDA8 en los tres umbrales de la NOM-020.

Al umbral vigente de 65 ppb las excedencias son el 11.25 % de los días-estación válidos: un clasificador que prediga siempre la clase mayoritaria acierta 88.75 % sin capturar nada. La exactitud queda descartada como métrica en la etapa de clasificación. El umbral de 51 ppb (31.96 %) está mucho mejor balanceado y conviene como escenario principal de modelado.

5. Criterios de aceptación

- OK Frecuencias horarias suman al total
- OK Frecuencias diarias suman al total
- OK `excede_anio_1` = 11.25 % sobre válidos (obtenido 11.25 %)
- OK Ningún pastel con más de 3 sectores